



Introduktion til diskret matematik og algoritmer: Exam April 8, 2026

Submission: Please write your solutions with ample margins on all sides, and make sure your handwriting is legible. *Start your solution of every new problem on a new page. Please mark every page with your name, exam number or something else that uniquely identifies your exam*, so that it is easy to see for every page which exam submission it is part of.

Exposition: Please try to be precise in your solutions and refrain from vague statements. As a rule, unless otherwise instructed you should not just state an answer, but always make sure to also *explain why* the answer is what it is. Provide clear references to any facts in the course literature used. *Write your solutions in such a way that a fellow student of yours could read, understand, and verify your solutions.*

Collaboration: All problems should be solved individually. No communication or collaboration is allowed, and solutions will be checked for plagiarism.

Reference material: Textbooks and handwritten notes (including lecture notes) are allowed. Other typewritten material, including (but not limited to) problem sets or previous exams with solutions, is not allowed. Please note that you deviate from definitions and algorithm descriptions in the course material at your own risk! If, however, slight variations of algorithms were presented in the textbooks and/or in class, such minor details do not matter.

Grading: A score of 220 points is guaranteed to be enough to pass the exam.

About the problems: Note that the problems are of quite different types, and are *not sorted in increasing order of difficulty*. Please read through the whole exam first, before you start working on any single problem, so that you can plan which order of dealing with the problems makes most sense to you. Note that this is a fairly large exam, and you can get a top grade without solving all problems. Also, partial answers to problems can sometimes give substantial amounts of points. **Good luck!**

- 1 (60 p) Solve the following problems using techniques that we have learned during the course. Please make sure to provide not just intuition but also formal proofs.

1a Prove that if $x_1, x_2, \dots, x_n$ are non-negative real numbers, then it always holds that

$$(1 + x_1)(1 + x_2) \cdots (1 + x_n) \geq 1 + x_1 + x_2 + \cdots + x_n.$$

1b Prove that every integer $n \geq 12$ can be written as $4a + 5b$ for a and b non-negative integers.

- 2 (60 p) As Jakob was preparing this year's instalment of the IDMA course in early 2026, he realized that he had been teaching discrete mathematics and algorithms in Copenhagen for 6 years. For some reason that is not so easy to explain, this made Jakob wonder what the largest positive integer n is such that $26!/6^n$ is also an integer. Could you help Jakob figure this out? Jakob does not really trust calculators, so he would very much like to see a clean and simple solution by hand that just uses what we learned during the IDMA course.

3 (80 p) In this problem we consider linked lists. The records in our list datatype have four attributes:

- **data**: the data stored in the record (which we will not care about, as usual);
- **key**: used for sorting the records in the list;
- **prev**: link to the previous element in the list;
- **next**: link to the next element in the list.

The lists are sorted in increasing order based on the key values in **key**. We assume that distinct records in our list have distinct key values, so that every value identifies a unique record in a list. To make the coding easier, every list is initialized with two dummy records **head** and **tail** that do not contain any data but are such that

- **head.key** = $-\text{INFINITY}$, **head.prev** = **NULL**, and **head.next** = **tail**;
- **tail.key** = $+\text{INFINITY}$, **tail.prev** = **head**, and **tail.next** = **NULL**.

This ensures that **head** will always be the first element in the list since $-\text{INFINITY} = -\infty$ is smaller than any key value and that **tail** will always be last since $+\text{INFINITY} = +\infty$ is larger than any key value.

Below you find four pseudocode methods for operating on the list. Please identify which method:

- (a) **Inserts** an element identified by a key value in the list.
- (b) **Removes** an element identified by a key value from the list.
- (c) **Looks up and returns** an element identified by a key value from the list.

Note that there is one spoiler piece of code that does none of the above. All four pieces of code start with exactly the same while loop, and all of them have a return statement regardless of whether this is needed or not (since it would be too easy to distinguish the methods otherwise).

You receive 10 points for each correct match and 10 more points for a brief but convincing explanation why the piece of code does what you claim it does (or, for the spoiler piece of code, why it does something else and why this does not make any sense).

```
METHOD1 (record)
    current := head
    while record.key > current.key {
        current := current.next
    }
    if record.key == current.key {
        previous      := current.prev
        current       := current.next
        previous.next := current
        current.prev  := previous
    }
    return record
```

```

METHOD2 (record)
    current := head
    while record.key > current.key {
        current := current.next
    }
    if record.key != current.key {
        current := NULL
    }
    return current

METHOD3 (record)
    current := head
    while record.key > current.key {
        current := current.next
    }
    if record.key >= current.key {
        previous      := current.prev
        current.prev  := previous.next
        previous      := previous.next
        current.next  := previous
    }
    return current

METHOD4 (record)
    current := head
    while record.key > current.key {
        current := current.next
    }
    if record.key < current.key {
        previous      := current.prev
        previous.next := record
        record.prev   := previous
        record.next   := current
        current.prev  := record
    }
    return record

```

- 4 (60 p) For each of the propositional logic formulas below, determine whether it is a tautology or not. If the formula is not a tautology, show how to add a single connective $\wedge$, $\vee$, or $\neg$ to make it into a tautology. Please make sure to justify your answers (e.g., by presenting truth tables, or by using rules for rewriting logic formulas that we have learned in class).

4a $((p \rightarrow q) \rightarrow r) \vee ((\neg p \wedge \neg r) \vee (q \wedge \neg r))$

4b $(p \rightarrow (q \vee r)) \leftrightarrow ((p \vee q) \vee (p \wedge r))$

5 (70 p) In this problem we wish to study heaps and how they can be used for sorting.

- 5a Run heapsort by hand to sort the array $A = [11, 3, 19, 5, 17, 2, 13, 7]$ in increasing order and give a detailed explanation of the first few steps of the algorithm. Specifically, show how the heap is built and how the first two numbers are sorted into their correct final positions. Make sure to explain what calls to the heap are made from the sorting algorithm and what comparisons are made, and illustrate how the heap and array have changed at the end of every “heapify” or “bubble” call (after all recursive subcalls have been taken care of).

Remark: Please note that there is no need to present the entire execution of the sorting algorithm. Just showing the initial steps as described above is fully sufficient.

- 5b Jakob thinks he has figured out a nifty way of finding the *minimal* element in a max-heap by calling the following recursive algorithm with index $i = 1$:

```
MAX-HEAP-FIND-MIN (A, i)
  if (2 * i > A.heapsize) {
    smallest := i
  }
  else {
    smallest := 2 * i
  }
  if (2 * i + 1 <= A.heapsize AND A[2 * i + 1] < A[smallest]) {
    smallest := 2 * i + 1
  }
  if (smallest == i) {
    return (A[i])
  }
  else {
    return (MAX-HEAP-FIND-MIN (A, smallest))
  }
```

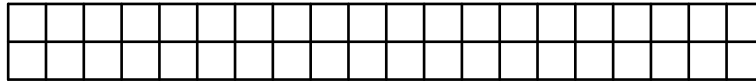
Is Jakob right that this will always return the smallest element (assuming that A is a correct max-heap)? Please provide either a proof of correctness or a counterexample where the input is a max-heap but MAX-HEAP-FIND-MIN fails to find the minimal element.

6 (80 p) In this problem we wish to count the number of ways of tiling a $2 \times n$ grid (as in Figure 1a) with 2×1 tiles so that every cell in the grid is covered by exactly one tile. There is 1 such tiling for a 2×1 grid (Figure 1b); exactly 2 different tilings for a 2×2 grid (Figures 1c and 1d); and exactly 3 different tilings for a 2×3 grid (Figures 1e, 1f, and 1g).

- 6a How many different tilings are there for a 2×4 grid? Make sure to explain how you compute this number so that we can follow your reasoning.

- 6b How many different tilings are there for a $2 \times n$ grid for a positive integer n ?

Hint: Find a nice formula for this number in terms of concepts we learned in class.



(a) Example 2×20 grid.



(b) Only tiling of 2×1 grid.



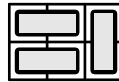
(c) First tiling of 2×2 grid.



(d) Second tiling of 2×2 grid.



(e) First tiling of 2×3 grid.



(f) Second tiling of 2×3 grid.



(g) Third tiling of 2×3 grid.

Figure 1: Tilings of $2 \times n$ grids in Problem 6.

7 (80 p) This problems is about relations in general and graphs in particular.

7a Say that the relation $E(u, v)$ holds for two vertices u and v in a directed graph if there is an edge from u to v . Present the matrix representation of E for the graph in Figure 2.

7b Take the reflexive closure R of the relation E in Problem 7a and present the matrix representation of this relation.

7c Take the transitive closure T of the relation R in Problem 7b and present the matrix representation of this relation.

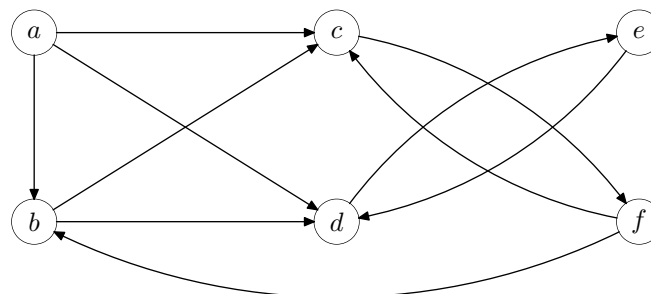


Figure 2: Graph for Problem 7

- 7d** Now consider a new relation $S(u, v)$ where we define $S(u, v)$ to hold for two vertices u and v precisely when $T(u, v)$ and $T(v, u)$ both hold for the relation T defined as in Problem 7c.

This gives a particular type of relation that we discussed in class. What is this type of relation called? Explain, briefly but clearly, why we are guaranteed to always get this type of relation when performing the steps described in Problems 7b, 7c, and 7d.

Also, this relation S groups the vertices of the graph in a particular way. What is the technical term that we use for such groups of vertices in a graph?

Remark: Note that you do *not* have to present any matrix representation for this relation S .

- 8** (100 p) Suppose that we have a universe of n elements and are interested in counting the number of multisets of size r . Jakob claims to have figured out that the number of such multisets is $\binom{n+r-1}{n-1}$. His idea is to prove this by using the following algorithm to translate any given size- r multiset M of $[n]$ to a standard subset S of $[n + r - 1]$ of size $n - 1$:

```
MULTISET2SET (M)
  elem := 1
  S := empty-set
  for i := 1 upto n - 1 {
    c := number of copies of element i in M
    for j := 1 upto c {
      elem := elem + 1
    }
    S := S union {elem}
    elem := elem + 1
  }
  return S
```

- 8a** Suppose that we have $n = 5$ elements in the universe and are interested in multisets of size $r = 6$. Describe what subset S is returned when the algorithm **MULTISET2SET** above is run with the multiset $M = [1, 1, 2, 5, 5, 5]$ as input. Make sure to explain, briefly but clearly, how this set is computed.

- 8b** Is Jakob's new algorithm **MULTISET2SET** correct? Can you prove that the algorithm provides a translation that shows that the number of multisets of size r from a universe of size n is exactly $\binom{n+r-1}{n-1}$? Or if there is an error in Jakob's algorithm, then what is it?

Hint: A proof of correctness would need to go through essentially the same kind of steps that we did in class. If Jakob's method is in fact incorrect, then just one small counterexample is sufficient.

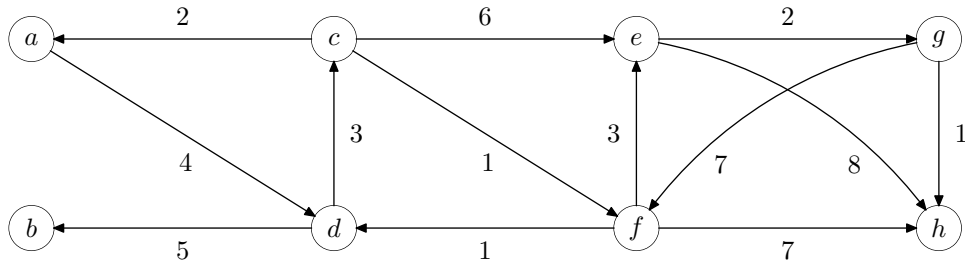


Figure 3: Graph for Problem 9

8c Consider the original translation from multisets of size r from a universe of size n to subsets of size r from a universe of size $n + r - 1$ that we used in class, and that we proved to be correct. For fixed positive integers n and r , let us say that the Boolean predicate $T_{n,r}(M, S)$ is true if the multiset M can be translated to the standard subset S by the method we studied in class, and that $T_{n,r}(M, S)$ is false otherwise.

Please indicate for all alternatives below whether $T_{n,r}$ satisfies this property or not. For each property, you get points for the correct answer, a deduction for a wrong answer, and no change in your score if you do not indicate an answer for a specific property.

1. $T_{n,r}$ describes a relation.
2. $T_{n,r}$ describes a function.
3. $T_{n,r}$ is surjective.
4. $T_{n,r}$ is reflexive.
5. $T_{n,r}$ is a partial order.
6. $T_{n,r}$ is injective.
7. $T_{n,r}$ is symmetric.
8. $T_{n,r}$ is an equivalence relation.

9 (120 p) Consider the graph in Figure 3 and the following ordered sequences listing the vertices in this graph:

1. a, d, b, c, e, f, g, h .
2. a, d, c, f, b, e, g, h .
3. a, d, c, b, e, g, h, f .
4. a, d, b, c, e, g, f, h .

For each of the sequences above, determine whether it can be the result of:

- (a) a breadth-first search with vertices listed in order of visits;
- (b) a depth-first search with vertices listed in order of discovery;
- (c) a shortest-path computation with vertices listed in the order they are removed from the priority queue.

Each sequence above is the output of at most one of the algorithms, but since there are four sequences there could be a sequence that cannot be produced by any of the algorithms.

Partial credit is given for matching algorithms and vertex sequences correctly. For full credit, you need to also give a brief overview of how and why the algorithms process the vertices in the given order (such as explaining the order of recursive calls, edges processed, or similar), or why none of the proposed algorithms could yield the sequence in question, but you do not have to provide detailed information about any auxiliary data structures used in the algorithms.